Invisible Formatting, Visible Effects: Whitespace Changes Flip LLM Outputs More Often Than Punctuation Changes

NeuriCo

Abstract

Text reaches language models with formatting the user never sees: Windows line endings, no-break spaces copied from web pages, trailing spaces, and typographic quotes and dashes from word processors. We ask whether such meaning-preserving edits change model outputs, and which kind matters more. We apply ten invisible whitespace and punctuation edits to three classification tasks and two code-generation tasks. We run 11 open models from 0.5B to 14.8B parameters with greedy decoding (239,616 generations), and we compare every edit against a byte-identical re-run of the same prompt. A single invisible edit flips 4.9% of classification labels (95% CI 4.6–5.1%), while the identical re-run flips 0.02%. Contrary to our hypothesis, whitespace edits flip 7.1% of labels and punctuation edits only 2.2%. Whitespace is more disruptive for every model and on both task types, and it stays more disruptive when we match the number of prompt tokens the edit changes. Models under 3B parameters flip labels 2.2× as often as larger ones (CI 2.0–2.4). For code, 36% of generations change text under an edit, but only 5.1% change their unit-test outcome, about the same rate as classification label flips. Net accuracy moves by less than 1 percentage point per model, because flips go in both directions and concentrate on items the model already gets wrong. Formatting therefore adds variance rather than bias. Practitioners should normalise whitespace before inference and report results over several formattings.

1 Introduction

Two prompts can look the same to a person and still differ in bytes. A review pasted from a web page carries no-break spaces (U+00A0) instead of ordinary spaces. A file saved on Windows ends its lines with `\r\n`. A word processor replaces straight quotes with curly ones and hyphens with dashes. None of these edits changes the meaning of the text, and most are invisible on screen. Our main question is simple: **do these invisible edits change what a language model outputs?**

The answer matters for anyone who measures or deploys LLMs. If invisible formatting changes outputs, then benchmark numbers contain a hidden formatting term, A/B tests of prompt variants are confounded, and the same input behaves differently depending on the editor or operating system that produced it. Small on-device models are a particular concern, because practitioners often assume they are brittle but rarely know by how much.

Prior work shows that LLMs are sensitive to prompt design. Changing separators, casing or field names in a few-shot template can move accuracy by tens of points Sclar et al. [2024], single-prompt evaluations give unstable rankings Mizrahi et al. [2024], and sensitivity varies across models and tasks Zhuo et al. [2024]. However, these studies perturb *visible* template structure. Character-level robustness work studies typos and adversarial edits Belinkov and Bisk [2018], Pruthi et al. [2019], Wang et al. [2021], which a reader can also see. We lack a controlled comparison of edits that a reader cannot see, one that separates whitespace from punctuation, spans model sizes within

Preprint.

families, covers both classification and code, and measures each effect against the noise floor of inference itself.

We fill this gap with a controlled study. We define five whitespace edits (trailing spaces, CRLF line endings, double spaces, trailing newlines, and no-break spaces or tabs) and five punctuation edits (curly quotes, curly apostrophes, Unicode dashes, the ellipsis character, and all four combined). We apply each edit to every item of SST-2, AG NEWS, BOOLQ, HUMANEVAL and MBPP, and we run 11 open instruction-tuned models from the QWEN2.5, QWEN3 and LLAMA-3.X families, ranging from 0.5B to 14.8B parameters. Every perturbed output is compared with the same model's output on the canonical prompt. Every item also gets a *control* condition: the byte-identical canonical prompt placed at another batch position, which measures how often outputs change for no reason at all. Finally, we pass every prompt through each model's chat template and tokenizer to check whether the edit survives and how many tokens it changes.

Figure 1 summarizes the main result. A single invisible edit flips 4.9% of classification labels on average, more than 200 times the 0.02% control rate. We expected punctuation to matter more than whitespace. The opposite holds: whitespace edits flip 7.1% of labels and punctuation edits 2.2%, and whitespace is worse for all 11 models. Part of the gap comes from token footprint, because a double-space or no-break-space edit re-tokenizes every word. Even at a matched footprint, though, whitespace is worse: a trailing `\n\n` (about one changed token) flips 3.4% of labels, while a Unicode dash (1.3 tokens) flips 1.3%. Models under 3B parameters flip 2.2× as often as larger models on classification, but code shows no size effect. Code text changes in 36% of generations, yet only 5.1% of programs change their test outcome.

Our contributions are:

- We propose a taxonomy of ten invisible formatting edits, split into whitespace and punctuation families, with code-safe variants that leave programs valid.
- We conduct a controlled study over 11 open models, 5 tasks and 239,616 greedy generations, with a byte-identical re-run as the noise floor and an exact replication in a separate engine.
- We show that whitespace edits are 2–5× more disruptive than punctuation edits, that small models are about 2× more label-sensitive, and that most code changes are stylistic rather than functional.
- We trace the effect to the token level. Edits that the chat template removes, such as trailing newlines under LLAMA-3.X templates, have zero effect, and divergence grows with the number of changed tokens without fully explaining the whitespace–punctuation gap.

Section 2 reviews related work, section 3 describes our setup, section 4 reports results, and section 5 discusses mechanisms, implications and limitations.

2 Related Work

Prompt-format sensitivity. A growing body of work shows that LLM outputs depend on prompt details that should not matter. Zhao et al. [2021] and Lu et al. [2022] show that the choice and order of few-shot examples can swing accuracy from near chance to near state of the art. Sclar et al. [2024] vary separators, casing and spacing between template fields and find accuracy differences of up to 76 points for LLaMA-2-13B. Mizrahi et al. [2024] show that model rankings change across paraphrased instructions and argue for multi-prompt evaluation. Zhuo et al. [2024] measure instance-level prompt sensitivity across models and link it to model confidence. These studies perturb *visible* structure that a prompt designer chooses. Unlike them, we hold the visible prompt fixed and change only characters a reader cannot see or barely notices, and we separate whitespace from punctuation edits.

Character-level robustness. Neural NLP models are known to be fragile under character noise. Belinkov and Bisk [2018] show that synthetic and natural typos break machine translation, Pruthi et al. [2019] show that single-character adversarial edits fool text classifiers, and toolkits such as TextFlint Wang et al. [2021] bundle many such transformations. These edits are visible typos or adversarially chosen changes. Our edits are neither: they are the benign byte-level variation that ordinary software introduces, and none of them changes how the text reads.

Tokenization. Byte-level BPE tokenizers Sennrich et al. [2016], Radford et al. [2019] fold a leading space into the following word token, so `"word"` and `" word"` map to different ids, and an unusual

Family	Condition	Edit	Applies to
Control	`control`	Byte-identical to canonical, different batch position	all
Whitespace	`trailing_space`	Append a space to every line	all
	`crlf`	Replace `\n` with `\r\n`	all
	`double_space`	Replace every single inter-word space with two spaces	all (code: prose only)
	`trailing_newlines`	Append `\n\n` to the message	all
	`nbsp`	Replace inter-word spaces with no-break space U+00A0	classification
	`tabs`	Replace 4-space indentation with a tab	HUMANEVAL
Punctuation	`curly_quotes`	`"x"` → “x”	all (code: prose only)
	`curly_apostrophe`	’ → ’ / ‘	all (code: prose only)
	`unicode_dash`	- → en dash, – → em dash, digit ranges → en dash	all (code: prose only)
	`ellipsis_char`	... → U+2026 (…)	40 cls / 9 code prompts
	`punct_all`	All four punctuation edits combined	all (code: prose only)

Table 1: The ten invisible formatting edits plus the identical-prompt control. “Prose only” means the edit touches the instruction, docstrings and comments but never code tokens or doctest lines, so programs stay valid.

space character changes the token of every word it precedes. This suggests a mechanism for whitespace sensitivity. We measure it directly by applying each model's chat template and tokenizer and counting the token positions an edit changes. We find that this token footprint explains part, but not all, of the gap between whitespace and punctuation.

Nondeterminism of inference. Greedy decoding is often treated as deterministic, but batched GPU inference with engines such as vLLM Kwon et al. [2023] can change outputs depending on batch composition, because floating-point reductions are not associative Atil et al. [2024]. Any sensitivity study that ignores this floor attributes noise to the perturbation. We include a byte-identical control condition for every item and report all effects against it. On code generation, this floor alone changes 6.2% of outputs.

3 Method

3.1 Problem formulation

Let f_m be model m with greedy decoding and let x_i be the canonical prompt for item i. A perturbation p is a string edit that preserves meaning and is invisible or nearly invisible when rendered. It produces $p(x_i)$. We measure divergence relative to the model's own canonical answer, not relative to the gold label:

$$D_m(p) = \frac{1}{|\mathcal{I}_p|} \sum_{i \in \mathcal{I}_p} \mathbf{1}\big[g(f_m(p(x_i))) \neq g(f_m(x_i))\big], \tag{1}$$

where g maps a raw output to the quantity of interest (a parsed label, the extracted code, or a unit-test outcome) and $\mathcal{I}_p = \{i : p(x_i) \neq x_i\}$ is the set of items the edit actually changes. The `control` condition sets p to the identity but places the prompt at a different position in the batch, so $D_m(\texttt{control})$ is the noise floor of inference.

3.2 Perturbations

Table 1 lists the edits. The whitespace family has four edits shared by all tasks, plus no-break spaces (`nbsp`) for classification or tab indentation (`tabs`) for code. The punctuation family replaces ASCII quotes, apostrophes, dashes and ellipses with their typographic Unicode forms, separately and all together (`punct_all`). We wrote the task instructions to contain double quotes, an apostrophe (“Don't”) and a spaced hyphen, so that every punctuation edit changes every prompt. The exception is `ellipsis_char`, which applies only to the 40 classification and 9 code prompts that contain .... Edits that leave a prompt unchanged are excluded from all rates.

Code prompts must stay valid Python. We apply `double_space` and all punctuation edits only to natural language: the instruction, docstrings and comments, which we locate with Python's `tokenize` module. Doctest lines (`>>>` and their expected output) are left untouched. Layout edits (`trailing_space`, `crlf`, `tabs`, `trailing_newlines`) apply to the whole message.

3.3 Tasks and models

Classification. We sample 400 items each (seed 42) from the GLUE Wang et al. [2018] validation split of SST-2 Socher et al. [2013], the test split of AG NEWS Zhang et al. [2015] and the validation split of BOOLQ Clark et al. [2019] (passages of at most 180 words). Each prompt is a zero-shot instruction such as `Classify the sentiment of the movie review below as "positive" or "negative".` followed by `Don't explain - answer with one word only.` and the input. We parse the earliest label keyword in the output, map synonyms, and assign `other` to anything else.

Code generation. We use all 164 HUMANEVAL problems Chen et al. [2021] and the 300-problem MBPP test split Austin et al. [2021]. HUMANEVAL prompts ask the model to complete the given function and return it in one fenced block. MBPP prompts give the task text and its three `assert` tests. We extract the first fenced block that contains a `def` and normalize CRLF and trailing whitespace. We then run the unit tests in an isolated subprocess (`python -I`, 2 GB memory limit, 20 s timeout).

Models. We run 11 open instruction-tuned models: QWEN2.5 0.5B, 1.5B, 3B, 7B and 14B Qwen Team [2024]; QWEN3 0.6B, 4B and 8B Yang et al. [2025], with thinking mode disabled; and Llama-3.2 1B and 3B and Llama-3.1 8B Grattafiori et al. [2024]. Most checkpoints are AWQ 4-bit Lin et al. [2024]. Llama-3.2-1B, QWEN3-0.6B and QWEN3-8B run in bf16. All models run in vLLM 0.32 Kwon et al. [2023] on NVIDIA RTX A6000 GPUs with temperature 0, using at most 12 new tokens for classification and 512 for code. We call the four models under 3B parameters *small* and the other seven *large*. Each model sees 19,968 prompts (1,664 items $\times$ 12 conditions), for 239,616 generations in total including a replication run. We had no commercial API access, so the study covers open-weight models only.

3.4 Metrics

We report four divergence metrics, each an instance of equation 1: (1) **label flip**, where the parsed classification label differs; (2) **code changed**, where the normalized extracted code differs; (3) **pass/-fail flip**, where the unit-test outcome differs; and (4) **raw text changed**, where the output string differs. We also report accuracy against gold labels and tests. For every model and prompt, we apply the chat template and tokenizer and record whether the token sequence changes and how many positions differ (Δtok, computed with `difflib`).

3.5 Statistical analysis

All metrics are paired at the item level. We compute 95% confidence intervals with a bootstrap over items (2,000 resamples). For the small/large ratio, the bootstrap resamples the same items for all models. For each of the 110 model $\times$ condition cells, we test perturbation against control with an exact McNemar test on discordant items and apply Holm correction Holm [1979]. We fit GEE logistic models Liang and Zeger [1986] with exchangeable correlation clustered by item, of the form flip $\sim$ punctuation + small + task, with and without $\log(1 + \Delta\text{tok})$, using statsmodels Seabold and Perktold [2010].

Reproducibility checks. We re-ran QWEN2.5-0.5B in a separate engine process on a different GPU. All 19,968 outputs were byte-identical to the first run, so every rate for that model replicates exactly. Llama-3.x chat templates strip trailing whitespace from messages, so `trailing_newlines` leaves their token sequence unchanged. This gives a natural placebo: an edit to the raw string that the model never sees.

4 Results

We first compare each edit family against the noise floor (section 4.1). We then break results down by edit (section 4.2), model and size (section 4.3) and task (section 4.4), and we close with mediators of the effect (section 4.5).

Task kind	Metric	Control	Whitespace	Punctuation	All edits
Classification (1,200 items)	label flip	0.02	**7.07**	2.17	4.88 [4.61, 5.14]
	raw text changed	0.13	**11.7**	5.2	–
Code (464 problems)	code changed	6.2	**47.6**	22.6	35.7 [34.2, 37.4]
	pass/fail flip	0.61	**7.17**	2.79	5.08 [4.69, 5.51]

Table 2: Divergence from the canonical output (%), averaged over items, models and conditions, with bootstrap 95% CIs over items. Every edit family is far above the identical-prompt control, and whitespace (bold) exceeds punctuation on every metric. For classification flips, the punctuation − whitespace difference is -4.9 pp$_{[-5.2,\,-4.6]}$.

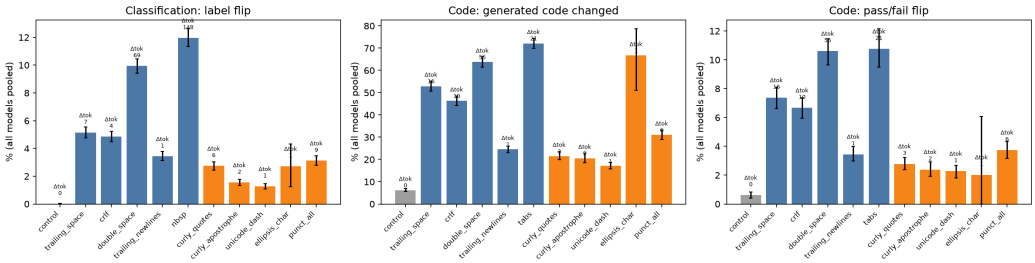


Figure 1: Divergence per edit, pooled over 11 models, with bootstrap 95% CIs. *(Left)*classification label flips. *(Center)*code text changes. *(Right)*code pass/fail flips. Blue bars are whitespace edits, orange bars punctuation edits, and grey is the identical-prompt control. Labels above the bars give Δtok, the mean number of changed prompt tokens. Every whitespace edit except `trailing_newlines` exceeds every punctuation edit on label and pass/fail flips.

4.1 Invisible edits change outputs far above the noise floor

Table 2 shows the main result. Re-running the byte-identical prompt flips 0.02% of classification labels. A single invisible edit flips 4.88% [4.61, 5.14], more than 200 times the floor. Perturbation vs. control is significant after Holm correction in 80 of 110 model $\times$ condition cells for label flips, 64 of 110 for code pass/fail flips and 89 of 110 for code text changes. Most non-significant cells are `ellipsis_char`, which touches few items, or punctuation edits on large models.

Is whitespace or punctuation more disruptive? We hypothesized that punctuation edits would cause more divergence (8–20%) than whitespace edits (3–8%). The data show the reverse. Whitespace edits flip 7.07% of labels and punctuation edits 2.17%, a difference of 4.9 pp$_{[4.6,\,5.2]}$. The same ordering holds for code text changes (47.6% vs. 22.6%) and code pass/fail flips (7.17% vs. 2.79%). In a GEE model clustered by item, the odds ratio for punctuation relative to whitespace is 0.28 for label flips ($p < 10^{-190}$) and 0.37 for code pass/fail flips.

4.2 Per-edit results

Figure 1 and table 3 break the effect down by edit. The two global whitespace edits are the most disruptive. `nbsp` flips 12.0% of labels and `double_space` 9.9%, and both rewrite dozens of tokens (Δtok of 148 and 69). Yet even the local whitespace edits, which change only a few tokens, beat every single punctuation edit: `trailing_space` flips 5.1% and `crlf` 4.9%, against 1.3–2.7% for the punctuation edits. Combining all four punctuation edits (`punct_all`, 3.1%) is still less disruptive than any whitespace edit except `trailing_newlines`. On code, `tabs` changes 72.1% of generations and flips 10.8% of pass/fail outcomes. Indentation and line-end characters in the prompt propagate into how the model formats its own code.

4.3 Per-model results and model size

Figure 2 and table 4 show results per model. Whitespace exceeds punctuation for all 11 models on label flips, code text changes and code pass/fail flips. Rates span two orders of magnitude, from 0% (Llama under `trailing_newlines`) to 39% (Llama-3.2-1B under `nbsp`). The one exception

		Classification		Code		
Family	**Condition**	Δtok	label flip	Δtok	code changed	pass/fail flip
Control	`control`	0	0.02	0	6.2	0.6
Whitespace	`trailing_space`	7.4	5.1	16.3	52.9	7.4
	`crlf`	4.3	4.9	10.1	46.3	6.7
	`double_space`	68.9	9.9	54.5	63.7	10.6
	`trailing_newlines`	0.7	3.4	1.5	24.5	3.5
	`nbsp / tabs`	147.7	**12.0**	20.8	**72.1**	**10.8**
Punctuation	`curly_quotes`	6.1	2.7	3.2	21.5	2.8
	`curly_apostrophe`	1.8	1.5	2.1	20.6	2.4
	`unicode_dash`	1.3	1.3	1.1	17.1	2.3
	`ellipsis_char`*	1.2	2.7	1.4	66.7	2.0
	`punct_all`	9.2	3.1	6.3	31.0	3.7

Table 3: Divergence (%) per edit, pooled over 11 models. Δtok is the mean number of prompt token positions changed after the chat template and tokenizer. The most disruptive edit in each column is in bold. *`ellipsis_char` applies to only 40 classification and 9 code items per model, so its CIs are very wide.

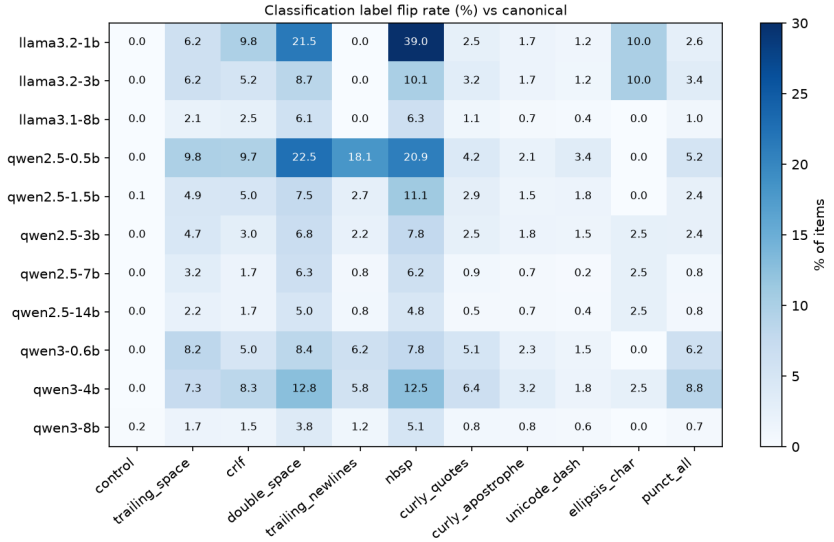


Figure 2: Classification label-flip rate (%) per model and edit. Llama-3.2-1B under `nbsp` reaches 39%, and QWEN2.5-0.5B under a single trailing `\n\n` reaches 18%. The Llama `trailing_newlines` column is exactly 0 because Llama chat templates strip trailing whitespace, so the model never sees the edit (a placebo). Punctuation columns stay below 1% for ≥7B models.

to the whitespace-first ordering is QWEN3-8B's raw classification *text*, which changes as often under punctuation as under whitespace. Most of these changes are capitalization of the same label (`positive` → `Positive`) and do not flip the label.

Are small models more sensitive? Table 5 answers this question. On classification, models under 3B parameters flip 2.22× as often as larger models [2.03, 2.41], and 2.88× within the QWEN2.5 family alone. The GEE model agrees (odds ratio 2.39 for small models, $p < 10^{-78}$). On code, there is no size effect. Large models change their code text as often as small ones (ratio 0.89 on the excess over control), and pass/fail flips do not differ significantly (1.14, CI includes 1; GEE odds ratio 1.16, $p = 0.14$). Figure 3 shows that sensitivity is not monotone in size: QWEN3-4B (7.4% label flips) is more sensitive than QWEN3-0.6B (5.6%). Part of this is a floor effect. QWEN3-0.6B labels 80% of AG NEWS items as "sports", and Llama-3.2-1B answers "no" to 95% of BOOLQ items. A near-constant predictor cannot flip.

		Classification label flip			Code (all edits)		Control	
Model	**Params (B)**	Whitespace	Punctuation	All	changed	pass/fail flip	cls flip	code changed
QWEN2.5-0.5B	0.5	**16.2**	3.7	**10.6**	47.8	6.3	0.0	5.2
QWEN3-0.6B	0.6	7.1	3.8	5.6	40.4	6.0	0.0	20.9
Llama-3.2-1B	1.2	15.3	2.0	9.4	29.3	3.7	0.0	15.1
QWEN2.5-1.5B	1.5	6.2	2.1	4.4	38.9	6.4	0.1	3.2
QWEN2.5-3B	3.1	4.9	2.0	3.6	45.1	**7.6**	0.0	4.3
Llama-3.2-3B	3.2	6.1	2.4	4.4	27.3	3.8	0.0	2.2
QWEN3-4B	4.0	9.4	**5.0**	7.4	28.6	3.9	0.0	0.9
QWEN2.5-7B	7.6	3.6	0.7	2.3	37.3	5.7	0.0	1.9
Llama-3.1-8B	8.0	3.4	0.8	2.2	29.5	4.3	0.0	2.4
QWEN3-8B	8.2	2.6	0.7	1.8	26.5	2.9	0.2	8.0
QWEN2.5-14B	14.8	2.9	0.6	1.9	**49.1**	6.0	0.0	7.1

Table 4: Divergence (%) per model. The horizontal rule separates small (<3B) from large (≥3B) models. The highest value in each perturbation column is in bold. Whitespace exceeds punctuation for every model. Small models dominate classification flips, while code text changes show no size trend. Control columns show batch-position noise, which is negligible for labels but reaches 21% of code outputs for QWEN3-0.6B.

Metric	**Small (<3B)**	**Large (≥3B)**	**Ratio small/large**
Classification label flip (all 11 models)	7.50	3.38	**2.22** [2.03, 2.41]
Classification label flip (QWEN2.5 only)	7.49	2.60	**2.88** [2.55, 3.32]
Code changed, excess over control	27.4	30.8	0.89 [0.84, 0.94]
Code pass/fail flip	5.51	4.83	1.14 [0.96, 1.37]

Table 5: Effect of model size (%). Ratios use a paired bootstrap over items with 95% CIs. Small models are 2–3× more label-sensitive (bold), but they are not more sensitive on code. The QWEN2.5-only row compares 0.5B/1.5B with 3B/7B/14B.

4.4 Task type

Table 6 reports divergence per task. Among classification tasks, flip rates track task ambiguity. The four-way AG NEWS task, with many borderline Business, Sci/Tech and World items, flips 8.5% of labels, about three times SST-2 (3.0%) and BOOLQ (3.1%). Code text is the most volatile output: 50.3% of HUMANEVAL and 27.7% of MBPP generations change. Measured by test outcome, however, code is about as affected as classification: 5.1% of programs change pass/fail status, compared with 4.9% of labels. In total, 31% of perturbed code generations changed their text without changing their test outcome.

4.5 What drives the effect?

The edit must survive the tokenizer. When the chat template and tokenizer produce an identical token sequence, outputs change only at the noise-floor rate: 0.02% label flips and 0.6% pass/fail flips. This happens for `trailing_newlines` under Llama templates, which trim trailing whitespace. When the token sequence changes, flips rise to 5.0% (classification) and 5.3% (code pass/fail). Whether an edit matters therefore depends on the serving template as much as on the model. The same `\n\n` flips 18% of QWEN2.5-0.5B labels and none of Llama's.

Larger edits matter more, but do not explain the gap. Divergence grows with the number of changed tokens (odds ratio 1.40 per unit of $\log(1 + \Delta\text{tok})$ for label flips). Adding this term to the GEE model moves the punctuation odds ratio from 0.28 to 0.45 for label flips and from 0.30 to 0.73 for code text changes, but punctuation remains significantly less disruptive. Matched pairs confirm this. `trailing_newlines` (Δtok $\approx$ 0.7) flips 3.4% of labels while `unicode_dash` (Δtok $\approx$ 1.3) flips 1.3%, and `crlf` (4.3 tokens) flips 4.9% while `curly_quotes` (6.1 tokens) flips 2.7%.

Flips land on uncertain items. Flips concentrate on items the model gets wrong under the canonical prompt. For ≥7B models, 7.8–9.3% of wrongly answered items flip, compared with 0.6–1.3% of correctly answered ones.

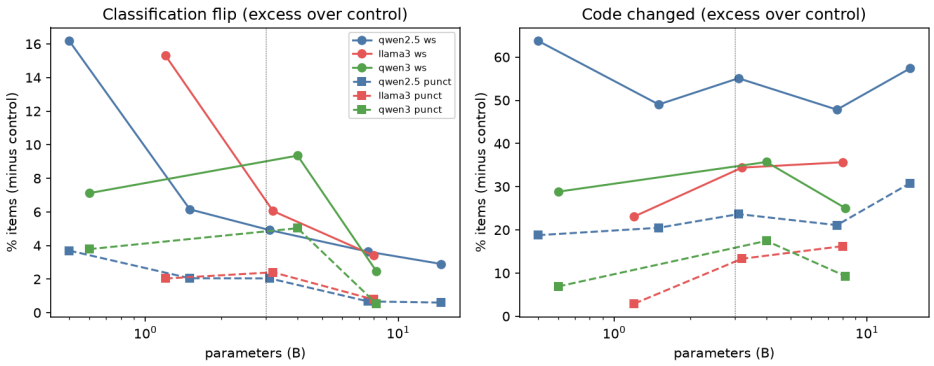


Figure 3: Excess divergence over the control vs. parameter count (log scale), per family. Solid lines are whitespace edits, dashed lines punctuation edits, and the dotted vertical line marks 3B. *(Left)*label flips fall with size for QWEN2.5 and LLAMA-3.X, while QWEN3 peaks at 4B. *(Right)*code text changes show no downward trend with size. Whitespace lies above punctuation for every family and size.

	SST-2	BOOLQ	AG NEWS	MBPP	HUMANEVAL
Label / pass-fail flip (%)	3.0	3.1	**8.5**	4.2	6.7
Code changed (%)	–	–	–	27.7	**50.3**

Table 6: Divergence per task, averaged over edits and models. The most affected task in each row is in bold. Functional flips are similar across task kinds, while raw code text is far more volatile than labels.

Accuracy barely moves. Flips are nearly symmetric: 42% go from right to wrong and 40% from wrong to right. The mean accuracy change per model and edit family stays within ± 1.3 pp, and most changes are within ± 0.7 pp. The largest drop is Llama-3.2-1B under whitespace edits (2.8 pp ↓on classification). Whitespace lowers code pass@1 by 0.56 pp on average. Invisible formatting therefore adds variance to individual predictions rather than a systematic bias.

5 Discussion

Hypotheses revisited. We started with four hypotheses. (H1) Invisible edits flip 5–15% of labels. This is *borderline*. The mean is 4.9% [4.6, 5.1], just below the range. Small models fall inside it (7.5%) and large models below it (3.4%), and individual cells range from 0% to 39%. (H2) Punctuation edits are more disruptive than whitespace edits. This is *refuted*. Whitespace edits flip 7.1% of labels, inside the predicted 3–8%, while punctuation edits flip only 2.2%, far below the predicted 8–20%. (H3) Models under 3B are 2–3× more sensitive. This is *supported for classification* (2.2× overall, 2.9× within QWEN2.5) but *not for code* (1.14×, not significant). (H4) Code is most affected, with 15–25% output change. This is *partly supported*. Code text changes more than predicted (36%), but functional flips (5.1%) match classification label flips (4.9%).

Why is whitespace worse than punctuation? We see two mechanisms. The first is token footprint. Byte-level BPE tokenizers attach the leading space to the next word, so a double space or a no-break space changes the token of every word in the prompt (69 and 148 changed positions on average), while curly quotes touch a handful of tokens. The second mechanism operates even at a matched footprint. A trailing \n\n changes the last tokens before the assistant turn, the position that most directly conditions the first answer token. CRLF changes every line boundary the model uses to parse the prompt's structure. Typographic quotes and dashes, by contrast, are common in web pretraining text, and models appear to treat them as near-synonyms of their ASCII forms. The GEE results support this reading: controlling for edit size shrinks the punctuation–whitespace gap but does not close it.

Why is code text so volatile but code correctness not? A 512-token program has many places where an equally valid choice can differ, such as a variable name, a loop form or a blank line. Greedy decoding commits to one branch, and a small change in the prompt can tip an early near-tie. Most of these alternatives are equally correct, so 31% of perturbed generations change text without changing test outcomes. The same reasoning explains the larger code noise floor: re-running an identical prompt in a different batch position already changes 6.2% of code outputs, up to 21% for QWEN3-0.6B, but only 0.02% of labels. Without this control, about a sixth of the code-change effect would be misattributed to formatting.

Implications for practice. First, whitespace normalization matters more than punctuation normalization. Converting `\r\n` to `\n`, replacing no-break spaces with spaces, collapsing repeated spaces and stripping trailing whitespace removes the most disruptive edits we tested. Second, a 2–5 point accuracy difference between two prompt variants on a small model can come from formatting alone. Evaluations should report results over several formattings and compare against an identical-prompt re-run. Third, chat templates silently normalize some formatting (Llama trims trailing whitespace) and not other formatting (QWEN2.5 keeps it), so the same raw input can be safe for one serving stack and fragile for another. Finally, because net accuracy barely moves, formatting is a source of variance in individual predictions rather than a systematic bias. This matters most for per-item consistency, such as when the same document is classified twice through different pipelines.

Limitations. We test only open models up to 14.8B parameters, since no commercial API was available, and frontier models may behave differently. Most checkpoints are AWQ 4-bit, and quantization may itself add sensitivity. bf16 QWEN3-8B is the least sensitive model, but it also differs in family and was served by a separate vLLM server, which raises its control noise. We use greedy decoding only, so we measure the deterministic component of the effect, not shifts in the sampling distribution. The edit set is a design choice. Two whitespace edits (`double_space`, `nbsp`) touch every word. Without them, whitespace flips 4.5% of labels (mean of `trailing_space`, `crlf` and `trailing_newlines`), still about twice the punctuation rate. Punctuation edits can only change the few quotes and dashes present, and a quote-heavy corpus could show larger effects. `ellipsis_char` rarely applied. In code prompts, punctuation edits touch only docstrings and comments, so they are smaller than in classification prompts. Near-constant predictions from Llama-3.2-1B on BOOLQ and QWEN3-0.6B on AG NEWS understate small-model sensitivity on those tasks. Finally, we use one prompt template per task, 400 items per classification task and 464 code problems, and other templates may be more or less fragile.

Broader impact. Our findings help practitioners build more reliable evaluations and pipelines. They could also inform adversarial manipulation of model outputs through invisible characters. Because the effects we observe are mostly symmetric and concentrated on uncertain items, simple input normalization is an effective defence.

6 Conclusion

We study whether invisible, meaning-preserving formatting edits change LLM outputs. Across 11 open models, five tasks and 239,616 greedy generations, a single invisible edit flips about 5% of classification labels (7.5% for models under 3B, 3.4% for larger ones), more than 200 times the rate of an identical re-run. Contrary to our hypothesis, whitespace edits (no-break spaces, double spaces, CRLF, trailing spaces and newlines) are 2–5$\times$ more disruptive than typographic punctuation, and token footprint explains only part of this gap. Code text changes in 36% of generations, but code correctness flips about as often as classification labels, and net accuracy barely moves. The key takeaway is that *whitespace is not neutral*: normalize it before inference, and evaluate prompts under several formattings against an identical-prompt baseline.

Future work should test frontier API models and design matched-token-budget edits that separate *what* changes from *how many* tokens change. Other directions include logit-margin analysis to test whether flips occur only when the top two labels are close, measuring distribution shift under sampling rather than greedy flips, and studying whitespace inside code bodies for code-repair tasks.

References

Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of "deterministic" LLM settings. *arXiv preprint arXiv:2408.04667*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Yonatan Belinkov and Yonatan Bisk. Synthetic and natural noise both break neural machine translation. In *International Conference on Learning Representations (ICLR)*, 2018.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of NAACL-HLT*, 2019.

Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Kung-Yee Liang and Scott L. Zeger. Longitudinal data analysis using generalized linear models. *Biometrika*, 73(1):13–22, 1986.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*, 12:933–949, 2024.

Danish Pruthi, Bhuwan Dhingra, and Zachary C. Lipton. Combating adversarial misspellings with robust word recognition. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024.

Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, 2010.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2013.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Xiao Wang, Qin Liu, Tao Gui, Qi Zhang, et al. TextFlint: Unified multilingual robustness evaluation toolkit for natural language processing. In *Proceedings of ACL-IJCNLP 2021: System Demonstrations*, 2021.

An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning (ICML)*, 2021.

Jingming Zhuo, Songyang Zhang, Xinyu Fang, Haodong Duan, Dahua Lin, and Kai Chen. ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

A Additional Experimental Details

Model checkpoints. Table 7 lists the exact checkpoints. All local models run in vLLM 0.32 on shared RTX A6000 GPUs. QWEN3-8B is served by a separate vLLM instance through its OpenAI-compatible API. Generation took about three hours in total, from 3–10 minutes for 0.5–1.5B models to about 1h40 for the 14B model.

Model	Checkpoint	Params (B)	Precision
QWEN2.5-{0.5,1.5,3,7,14}B	`Qwen/Qwen2.5-{...}-Instruct-AWQ`	0.5–14.8	AWQ-INT4
Llama-3.2-1B	`unsloth/Llama-3.2-1B-Instruct`	1.2	bf16
Llama-3.2-3B	`AMead10/Llama-3.2-3B-Instruct-AWQ`	3.2	AWQ
Llama-3.1-8B	`hugging-quants/Meta-Llama-3.1-8B-Instruct-AWQ-INT4`	8.0	AWQ
QWEN3-0.6B	`Qwen/Qwen3-0.6B` (thinking off)	0.6	bf16
QWEN3-4B	`Qwen/Qwen3-4B-AWQ` (thinking off)	4.0	AWQ
QWEN3-8B	`Qwen/Qwen3-8B` via vLLM server (thinking off)	8.2	bf16

Table 7: Model checkpoints. Small models (<3B) are QWEN2.5-0.5B, QWEN2.5-1.5B, Llama-3.2-1B and QWEN3-0.6B.

Prompt templates. The classification instructions are:

- SST-2: `Classify the sentiment of the movie review below as "positive" or "negative". Don't explain - answer with one word only. Review: ... Sentiment:`
- AG NEWS: the same format with the four topics `"World"`, `"Sports"`, `"Business"` or `"Sci/Tech"`.
- BOOLQ: `Read the passage and answer the question with "yes" or "no".` followed by the same answer-format line, the passage and the question.

For HUMANEVAL, the prompt reads: `Complete the following Python function. Don't change its signature - return the complete function ... in a single`

```python code block, without "explanations" or tests., followed by the function stub. For MBPP, the prompt contains the task text, Your code should pass these tests: and the three asserts. The double quotes, the apostrophe in "Don't" and the spaced hyphen ensure that every punctuation edit except ellipsis_char changes every prompt.

**Per-model code pass/fail flips.** Figure 4 shows the per-model pass/fail flip rate for code. As with classification, whitespace columns dominate. double_space flips 8–14% of outcomes for every model, and tabs reaches 21% for QWEN3-0.6B. Under Llama templates, trailing_newlines stays at the control level (0.2–1.3%), consistent with the placebo.

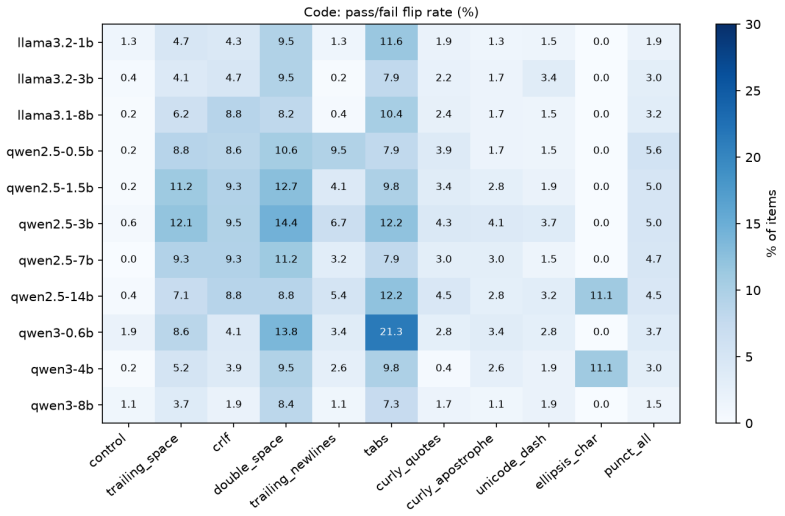



Figure 4: Code pass/fail flip rate (%) per model and edit, pooled over HUMANEVAL and MBPP. Whitespace edits produce the largest functional changes across models. The 11.1% ellipsis_char cells correspond to a single program out of 9.
```